

Curso prático de programação em Python

Aplicado à engenharia de defesa

QUANTUM STRATEGIC AI



Programação com Python

Aplicada à Engenharia de Defesa

Fundamentos da linguagem, do ambiente de trabalho ao primeiro miniprojeto integrador de apoio à decisão

AUTOR

Cristiano da Costa Alves

Programação com Python Aplicada à Engenharia de Defesa

Livro-texto de um curso prático de programação em Python, com carga de 40 horas, voltado à engenharia de defesa.

© 2026 Cristiano da Costa Alves.

Revisão Técnica: José Cassiano da Costa Alves

Especialista em ensino STEAM: Science, Technology, Engineering, Arts, Mathematics

Versão de referência da linguagem: Python 3.12.

Ambiente principal: Google Colab. Ambiente local alternativo: Anaconda/ `venv` e VS Code.

O código-fonte, os *notebooks* e os conjuntos de dados de apoio são distribuídos sob licença MIT em repositório público.

Esta é uma edição em desenvolvimento (versão de trabalho), destinada ao uso didático em cursos de programação. Erros e sugestões podem ser comunicados ao autor para incorporação em edições futuras.

Composto em L^AT_EX. Fontes: Palatino (texto) e Helvetica (títulos).

Apresentação

Este livro nasceu de uma necessidade concreta de sala de aula: a de um material enxuto que ensine a programar em Python no contexto da engenharia de defesa, sem o peso de uma obra dimensionada para um curso semestral. Ele foi concebido para um curso prático de 40 horas, organizado em quatro módulos, frequentado por um público adulto e heterogêneo. Programar, aqui, não é um fim em si: é a competência instrumental que destrava o trabalho técnico que vem depois – da análise de dados de sensores aos primeiros passos rumo a *machine learning* e inteligência artificial aplicados à defesa.

Os materiais existentes em português não atendem bem a essa realidade. De um lado, há livros introdutórios genéricos, que ensinam a linguagem sem nunca tocar em um problema de defesa. De outro, há obras técnicas extensas, dimensionadas para cursos longos, com três ou quatro vezes mais conteúdo do que cabe em 40 horas. Faltava um livro-texto enxuto, contextualizado na engenharia de defesa e calibrado para um curso curto e intensivo. É essa lacuna que este livro procura preencher.

A obra organiza-se em torno de um *miniprojeto integrador* – um pequeno sistema de apoio à decisão construído passo a passo ao longo dos quatro módulos, que culmina em um projeto final. Cada capítulo combina teoria concisa, código que se executa de imediato e exercícios graduados, pensados para o estudo entre um módulo e o seguinte. Os exemplos são ambientados em cenários reais de defesa nacional – da vigilância da Amazônia Azul ao monitoramento de fronteiras –, sempre no nível adequado a um curso de programação.

Escrevi para dois leitores ao mesmo tempo: aquele que nunca digitou uma linha de código e aquele que já programou em outra linguagem, como MATLAB, C ou VBA. Ao primeiro, ofereço um módulo zero de nivelamento, de autoestudo, a ser percorrido antes do primeiro módulo; ao segundo, um ritmo que não o entedia. A ambos, a convicção de que programar é uma habilidade que se adquire fazendo – e que vale o esforço.

O autor
2026

Como usar este livro

A obra cobre um curso de 40 horas, organizado em quatro módulos – de cerca de dez horas cada – que somam nove capítulos. Cada capítulo foi pensado em **três tempos**:

Antes do encontro

uma leitura preparatória curta, que situa os conceitos e prepara o terreno.

Durante o encontro

a exposição e a prática guiada, com o código que se executa e modifica em aula.

Após o encontro

os exercícios de consolidação, para fixar o conteúdo no intervalo entre um módulo e o seguinte.

Ao longo do texto, três caixas reaparecem com regularidade. Vale reconhecê-las desde já:

Contexto de defesa

Conecta o conceito que está sendo estudado a uma aplicação real no domínio da defesa e segurança nacional.

Armadilha comum

Aponta um erro frequente – de sintaxe, de lógica ou de método – e mostra como evitá-lo. Vale a pena ler antes de cometer o erro.

Boa prática

Recomendações de estilo, organização e arquitetura que separam o código que apenas funciona daquele que se pode manter, auditar e reaproveitar.

Cada capítulo encerra com um **resumo** dos conceitos principais, uma síntese das armadilhas comuns e **exercícios em três níveis**: *essencial* (fixação, com solução guiada no repositório), *tático* (aplicação a um problema semelhante) e *estratégico* (extensão criativa). Não é preciso resolver todos: os essenciais garantem o mínimo; os demais recompensam quem quer ir além.

Todo o código está disponível em um repositório público, com *notebooks* prontos para o Google Colab – um por capítulo –, executáveis no navegador, sem qualquer instalação. Recomendamos abrir o *notebook* do capítulo ao lado do texto e executar cada trecho de código à medida que ele aparece. Programação se aprende com as mãos no teclado.

Amostra gratuita

Esta amostra reúne os **Capítulos 1 a 3** (Módulo 1) da obra *Programação com Python Aplicada à Engenharia de Defesa*, oferecida para leitura e avaliação. O livro completo abrange nove capítulos, organizados em quatro módulos.

Os *notebooks* para o Google Colab de todos os capítulos disponíveis, o código-fonte e os dados de apoio são distribuídos sob licença MIT no repositório público do projeto.

Sumário

Apresentação	ii
Como usar este livro	iv
1 A Engenharia de Defesa, as Tecnologias Estratégicas e o Lugar da Programação	1
1.1 O ecossistema da defesa nacional	2
1.2 O lugar da programação na engenharia de defesa	4
1.3 Por que Python	5
1.4 Preparando o ambiente de trabalho	7
1.5 O miniprojeto integrador	10
1.6 O caminho à frente	12
Resumo do capítulo	13
Armadilhas comuns	13
Exercícios	14
2 Fundamentos da Programação em Python	16
2.1 Variáveis e tipos de dados	17
2.2 Operadores	19
2.3 Entrada e saída de dados	22
2.4 Estruturas condicionais	23
2.5 Estruturas de repetição	25
2.6 Funções: organizando e reutilizando código	27
2.7 Juntando tudo: um pequeno processador de telemetria	28
2.8 O caminho à frente	30

Resumo do capítulo	30
Armadilhas comuns	31
Exercícios	31
3 Estruturas de Dados Nativas	33
3.1 Listas: a estrutura de trabalho	34
3.2 Compreensão de listas	37
3.3 Tuplas: dados que não mudam	38
3.4 Dicionários: dados com rótulo	40
3.5 Conjuntos: unicidade e pertencimento	42
3.6 Escolhendo a estrutura certa	43
3.7 Miniprojeto: modelando os dados (Módulo 1)	44
3.8 O caminho à frente	46
Resumo do capítulo	47
Armadilhas comuns	47
Exercícios	48

A Engenharia de Defesa, as Tecnologias Estratégicas e o Lugar da Programação

© Objetivos do capítulo

Ao final deste capítulo, você será capaz de:

- situar a programação no contexto da engenharia de defesa e compreender por que ela é a *fundação* do trabalho técnico mais avançado;
- descrever, em linhas gerais, o ecossistema da defesa nacional – as Forças Armadas, a Base Industrial de Defesa e a Estratégia Nacional de Defesa – e identificar onde a programação se insere nesse ecossistema;
- justificar a escolha do Python como linguagem de trabalho para a engenharia de defesa;
- preparar o ambiente de trabalho, no Google Colab (principal) ou localmente (alternativa), e executar seu primeiro programa;
- reconhecer o miniprojeto integrador que será construído ao longo dos quatro módulos.

Este é um capítulo de orientação. Antes de escrever a primeira linha

de código “de verdade” – o que faremos no Capítulo 2, ao tratar dos fundamentos da linguagem –, é preciso responder a três perguntas que dão sentido a tudo o que vem depois: *em que mundo* estamos programando, *por que* programar e *com que ferramentas*. A primeira pergunta nos leva ao ecossistema da defesa nacional; a segunda, ao papel da programação na engenharia de defesa; a terceira, à escolha do Python e à preparação do ambiente. Ao final, apresentamos o fio condutor de todo o livro: o miniprojeto integrador.

1.1 O ecossistema da defesa nacional

A engenharia de defesa não opera no vácuo. Ela serve a um conjunto articulado de instituições, políticas e capacidades que, em conjunto, sustentam a soberania e a segurança do país. Conhecer esse pano de fundo é o que distingue um engenheiro que apenas programa de um engenheiro que programa *para a defesa* – alguém capaz de traduzir um problema operacional ou estratégico em código útil. Vejamos os quatro pilares desse ecossistema.

1.1.1 As Forças Armadas

A Marinha, o Exército e a Força Aérea são, simultaneamente, os principais *usuários* e os principais *demandantes* de tecnologia de defesa. São elas que operam navios, blindados, aeronaves, radares, sistemas de comunicação e de comando e controle, e é a partir de suas necessidades operacionais que nascem boa parte dos requisitos de engenharia. Um sistema de apoio à decisão para patrulha de fronteira, um modelo de previsão de manutenção de uma frota, uma ferramenta de análise de dados de sensores – todos respondem, em última instância, a uma necessidade operacional concreta.

Para o engenheiro de defesa, isso tem uma consequência prática: o software que escrevemos quase nunca é um exercício abstrato. Ele tem um *usuário* com uma *missão*, e essa missão impõe critérios de confiabilidade, rastreabilidade e robustez que nem sempre estão presentes em outros domínios.

1.1.2 A Base Industrial de Defesa (BID)

A Base Industrial de Defesa é o conjunto de empresas – públicas e privadas –, instituições de pesquisa e universidades que projetam, produzem e mantêm os produtos e serviços de defesa. É na BID que se desenvolvem desde sistemas de armas e plataformas até software embarcado, simuladores e soluções de análise de dados. Muitos leitores deste livro atuam exatamente aqui: como engenheiros, analistas e gestores de projetos na indústria de defesa.

A relevância para nós é direta. Cada vez mais, o valor de um produto de defesa está no *software* que o acompanha – no algoritmo que processa o sinal do radar, no modelo que estima o desgaste de um componente, na interface que apresenta a informação ao operador. Saber programar deixou de ser uma vantagem diferencial para se tornar uma competência básica de quem trabalha na BID.

1.1.3 A Política e a Estratégia Nacional de Defesa

Acima das instituições e das empresas estão os documentos que orientam o setor: a Política Nacional de Defesa (PND), que estabelece os objetivos, e a Estratégia Nacional de Defesa (END), que define *como* alcançá-los. É a END que articula a defesa ao desenvolvimento científico e tecnológico do país e que destaca a importância de *capacitar* recursos humanos e de *nacionalizar* tecnologias sensíveis – isto é, dominar internamente o conhecimento, em vez de depender de fornecedores externos.

Contexto de defesa

A ênfase da Estratégia Nacional de Defesa na autonomia tecnológica explica por que um curso voltado à defesa investe tempo em ensinar programação. Dominar o código é uma forma concreta de *nacionalizar* capacidade: quem sabe construir a ferramenta não depende de terceiros para mantê-la, auditá-la ou adaptá-la a uma nova necessidade.

1.1.4 As tecnologias estratégicas

Por fim, há um conjunto de tecnologias consideradas *estratégicas* pelo seu potencial de transformar a defesa: inteligência artificial, ciência de dados, simulação computacional, sistemas autônomos, guerra cibernética, sensoriamento e processamento de sinais, entre outras. O traço comum a todas elas é evidente: *todas* se materializam em software. Não há inteligência artificial sem programação; não há análise de dados sem código; não há simulação sem um modelo implementado em alguma linguagem.

É essa observação que nos conduz à segunda pergunta do capítulo.

1.2 O lugar da programação na engenharia de defesa

Se as tecnologias estratégicas se materializam em software, então a programação é a competência que dá acesso a todas elas. Neste curso, isso se traduz em um papel muito específico para a programação: ela é a **fundação computacional** sobre a qual o trabalho técnico mais avançado se apoia.

Contexto de defesa

Os temas técnicos mais avançados da engenharia de defesa – *machine learning*, inteligência artificial, simulação de sistemas complexos – *pressupõem* que se saiba programar. Um modelo de aprendizado de máquina é, antes de tudo, código Python que carrega dados, treina um algoritmo e avalia resultados. Sem uma base sólida na linguagem, o acesso a esses temas fica comprometido.

Vale insistir num ponto que organiza todo este livro: aqui, programar não é um fim em si mesmo. É uma *competência instrumental*. Este curso não pretende formar engenheiros de software profissionais em 40 horas – nem é disso que o engenheiro de defesa precisa. O que se busca é *autonomia computacional*: a capacidade de transformar um problema de defesa em um programa que o resolva, de ler e adaptar o código de outra pessoa, e de seguir aprendendo as ferramentas mais avançadas adiante.

Onde, na prática, a programação entra no trabalho do engenheiro de defesa? Em toda parte. Alguns exemplos, propositadamente variados:

- **Automação de tarefas repetitivas** – consolidar dezenas de planilhas de relatório em um único resumo, renomear e organizar arquivos, gerar documentos a partir de modelos.
- **Análise de dados** – extrair tendências de séries temporais de sensores, cruzar registros de ocorrências, produzir gráficos que apoiem uma decisão.
- **Prototipação de modelos** – testar rapidamente uma ideia de cálculo de alcance, de logística de meios ou de previsão de manutenção, sem depender de um sistema pronto.
- **Pequenas aplicações de apoio** – construir uma ferramenta com interface gráfica que um colega não programador consiga usar.

Boa prática

A melhor maneira de avaliar se você *realmente* aprendeu um conceito de programação é tentar aplicá-lo a um problema do seu próprio dia a dia profissional. Ao longo do livro, sempre que encontrar uma técnica nova, pergunte-se: “onde, no meu trabalho, isso seria útil?”. É essa transferência, e não a memorização da sintaxe, que transforma o curso em competência duradoura.

1.3 Por que Python

Existem dezenas de linguagens de programação, e muitas seriam tecnicamente adequadas. A escolha do Python para este curso – e para a engenharia de defesa em geral – não é arbitrária. Repousa sobre quatro razões.

1.3.1 Legibilidade e baixa curva de entrada

O Python foi projetado para ser legível. Sua sintaxe é próxima do inglês estruturado, dispensa muitos dos símbolos que tornam outras linguagens intimidantes para iniciantes e força, pela própria indentação, um código organizado. Para uma turma heterogênea, em que convivem alunos que nunca programaram com alunos vindos de MATLAB, C ou VBA, essa

baixa barreira de entrada é decisiva: permite a todos chegar rapidamente ao que importa – resolver problemas.

1.3.2 Um ecossistema científico maduro

Esta é, talvez, a razão mais forte. Em torno do Python cresceu o ecossistema mais completo que existe hoje para computação científica, análise de dados e inteligência artificial. Bibliotecas como NumPy (computação numérica), pandas (dados tabulares), Matplotlib e Seaborn (visualização) – todas estudadas neste livro – e, mais adiante, scikit-learn, TensorFlow e PyTorch formam uma cadeia contínua. O que se aprende aqui não se descarta depois: acumula-se.

Contexto de defesa

A predominância do Python na ciência de dados e na inteligência artificial não é um detalhe acadêmico. É a razão de a programação ser a porta de entrada para o trabalho técnico: o Python que você aprende aqui é, literalmente, a mesma linguagem com que se constroem os modelos de *machine learning* e de IA aplicados à defesa.

1.3.3 Gratuito, aberto e multiplataforma

O Python é software livre, sem custo de licença, e roda em Windows, Linux e macOS. Para o setor de defesa, em que a autonomia e a auditabilidade são valores, trabalhar com uma linguagem aberta – cujo funcionamento se pode inspecionar e cuja distribuição não depende de um único fornecedor – é uma vantagem estratégica, e não apenas econômica.

1.3.4 Python 3.12 como versão de referência

Adotamos, ao longo de todo o livro, o Python na versão 3.12. A escolha de fixar uma versão de referência tem um motivo prático: garante que todos os exemplos se comportem da mesma forma na sua máquina e na nossa. O Python 3.12 tem suporte oficial estendido por vários anos, o que assegura a validade dos exemplos por toda a vida útil deste livro.

Nota

Você não precisa instalar o Python 3.12 manualmente. O ambiente que adotaremos a seguir – o Google Colab – já oferece uma versão recente e configurada da linguagem, com as principais bibliotecas científicas prontas para uso. A versão exata pode variar ligeiramente, mas isso não afetará nenhum dos exemplos deste livro.

1.4 Preparando o ambiente de trabalho

Programar exige um lugar onde escrever e executar o código. Há, essencialmente, dois caminhos: trabalhar *na nuvem*, pelo navegador, sem instalar nada; ou montar um ambiente *local*, na sua própria máquina. Adotaremos o primeiro como principal e apresentaremos o segundo como alternativa.

1.4.1 Google Colab: o ambiente principal

O Google Colab (abreviação de *Colaboratory*) é um serviço gratuito que executa código Python diretamente no navegador. Não há nada a instalar: basta uma conta Google e uma conexão à internet. O Colab fornece um ambiente já pronto, com o Python e as bibliotecas científicas instaladas, e organiza o trabalho em *notebooks* – documentos que intercalam blocos de código executável com blocos de texto explicativo.

Contexto de defesa

A escolha do Colab é especialmente adequada a um curso curto e intensivo. Eliminar o atrito de instalação – que costuma consumir as primeiras horas de qualquer curso de programação e frustrar quem nunca configurou um ambiente – significa que todos chegam ao primeiro encontro prontos para programar. Cada capítulo deste livro acompanha um *notebook* pronto para o Colab.

Para começar, três passos bastam:

1. Acesse colab.research.google.com e entre com uma conta Google.

2. Crie um novo *notebook* (menu *Arquivo* → *Novo notebook*). Você verá uma célula vazia.
3. Digite o código na célula e execute-a com `Shift+Enter`. O resultado aparece logo abaixo.

Um *notebook* é composto de células. As *células de código* contêm Python e podem ser executadas; as *células de texto* contêm anotações escritas em formato *Markdown*. Essa mistura de código, resultado e explicação no mesmo documento é o que torna o *notebook* um excelente ambiente de aprendizado – e, mais tarde, de comunicação de análises.

1.4.2 O primeiro programa

A tradição manda que o primeiro programa em qualquer linguagem apenas exiba uma mensagem na tela. Vamos respeitá-la, com uma pequena adaptação ao nosso domínio. Digite o trecho a seguir em uma célula do Colab e execute-o:

Listagem 1.1: Seu primeiro programa em Python.

```
1 # Primeiro contato com o Python
2 print("Olá, Engenharia de Defesa!")
3 print("Ambiente de trabalho pronto para o curso.")
```

A saída esperada é:

```
1 Olá, Engenharia de Defesa!
2 Ambiente de trabalho pronto para o curso.
```

A função `print` exibe na tela o que recebe entre parênteses. O texto entre aspas é o que chamaremos, no próximo capítulo, de *string* (cadeia de caracteres). A linha que começa com `#` é um *comentário*: o Python a ignora; ela existe apenas para quem lê o código.

Para confirmar que o ambiente faz mais do que exibir texto, façamos uma conta com sabor de defesa. A Amazônia Azul – a vasta área marítima sob jurisdição brasileira – tem cerca de 3,6 milhões de quilômetros quadrados. Quantos campos de futebol regulamentares (de aproximadamente 0,007 km² cada) caberiam nessa área?

Listagem 1.2: Uma primeira conta: a dimensão da Amazônia Azul.

```

1 # A Amazônia Azul em "campos de futebol"
2 area_amazonia_azul_km2 = 3_600_000      # área aproximada,
   em km^2
3 area_campo_km2 = 0.007                  # área de um
   campo, em km^2
4
5 quantidade = area_amazonia_azul_km2 / area_campo_km2
6 print("Cabem cerca de", round(quantidade), "campos de
   futebol.")

```

```

1 Cabem cerca de 514285714 campos de futebol.

```

Não se preocupe ainda com os detalhes da sintaxe – variáveis, operadores e funções são o assunto do Capítulo 2. Por ora, basta perceber duas coisas: o ambiente funciona, e o Python permite, em pouquíssimas linhas, transformar um dado de defesa em uma informação compreensível. É exatamente esse poder que exploraremos ao longo do curso.

Armadilha comum

Em Python, o cifrão e a vírgula *não* são usados para separar milhares. Se você escrever `3, 600, 000` esperando “três milhões e seiscentos mil”, o Python entenderá três valores separados. Para legibilidade, use o sublinhado – `3_600_000` – que o Python ignora na leitura do número. E lembre-se: o separador decimal é o *ponto* (`0.007`), não a vírgula.

1.4.3 O ambiente local: uma alternativa

Embora o Colab seja o ambiente recomendado neste curso, em muitos contextos profissionais – sobretudo na defesa, onde o trabalho pode exigir que os dados *não* saiam da rede interna – você precisará programar localmente, na sua própria máquina. Apresentamos aqui, em linhas gerais, o caminho; o passo a passo detalhado, específico para cada sistema operacional, está no apêndice de instalação.

Há dois componentes a instalar:

O interpretador e as bibliotecas

A forma mais simples para quem está começando é instalar a distribuição **Anaconda**, que traz, em um único pacote, o Python e as principais bibliotecas científicas (NumPy, pandas, Matplotlib, entre outras). Para quem prefere um ambiente mais enxuto, a alternativa é instalar o Python oficial e criar um *ambiente virtual* com o módulo `venv`, instalando apenas as bibliotecas necessárias.

O editor de código

Recomendamos o **VS Code** (Visual Studio Code), um editor gratuito, multiplataforma e com excelente suporte ao Python. Ele oferece destaque de sintaxe, sugestões de código e a possibilidade de executar os mesmos *notebooks* que usamos no Colab.

Boa prática

Mesmo que você tenha o Python instalado globalmente, crie um *ambiente virtual* para cada projeto. Um ambiente virtual isola as bibliotecas de um projeto das de outro, evitando que a atualização de um pacote em um trabalho quebre outro. Em defesa, onde a *reprodutibilidade* de um resultado pode ser auditada, saber exatamente quais versões de quais bibliotecas produziram um determinado resultado deixa de ser conveniência e passa a ser requisito.

Nota

Neste curso, você não precisa escolher de imediato entre o Colab e o ambiente local. Comece pelo Colab – é o caminho de menor atrito – e migre para o ambiente local quando e se a sua necessidade profissional pedir. Todos os exemplos do livro rodam, sem alteração, nos dois ambientes.

1.5 O miniprojeto integrador

Há um fio que costura os nove capítulos deste livro: o **miniprojeto integrador**. Em vez de apresentar cada conceito de forma isolada, construiremos, do início ao fim do curso, um pequeno sistema de apoio à decisão – algo

simples, mas completo e funcional. A cada módulo, o projeto ganha uma nova capacidade, sempre usando o conceito que acabamos de estudar.

O escopo é deliberadamente contido, à medida das 40 horas do curso. Pense em um sistema de cadastro e análise de ocorrências de monitoramento – por exemplo, registros de detecções em uma área de vigilância – ou em um inventário de meios. A evolução, módulo a módulo, será a seguinte:

Módulo 1 – Modelagem dos dados

Representamos as informações do sistema usando as estruturas de dados nativas do Python (listas, dicionários). O projeto nasce como um conjunto organizado de dados em memória.

Módulo 2 – Persistência e organização

Os dados passam a ser gravados em arquivo e em um banco de dados SQLite, e o código é reestruturado em uma arquitetura orientada a objetos, mais limpa e manutenível.

Módulo 3 – Interface e cálculo

O sistema ganha uma interface gráfica em Tkinter, com a qual o usuário interage, e rotinas de cálculo numérico com NumPy.

Módulo 4 – Análise e decisão

Incorporamos a análise e a visualização de dados com `pandas` e `Matplotlib`, transformando os dados brutos em gráficos e indicadores que apoiam uma decisão. O sistema está completo.

Ao final do quarto módulo, você terá em mãos uma aplicação funcional – e, mais importante, terá *construído* cada parte dela, entendendo o porquê de cada escolha. Essa aplicação é a base direta do **projeto final** do curso: cada participante desenvolve uma variação própria, ambientada em um problema de seu interesse profissional. Quando pertinente, esse projeto pode até constituir o ponto de partida para um trabalho de maior fôlego.

Contexto de defesa

Um “sistema de apoio à decisão” não precisa ser grandioso para ser útil. Em muitos contextos de defesa, a diferença entre uma decisão

bem informada e um palpite está em uma ferramenta simples que organiza dados dispersos e os apresenta de forma clara. Aprender a construir essa ferramenta, do dado bruto ao gráfico, é um dos objetivos centrais deste livro.

1.6 O caminho à frente

Encerramos a orientação com um mapa do percurso. Os nove capítulos distribuem-se em quatro módulos, de cerca de dez horas cada:

Módulo	Conteúdo	Horas
I – Fundamentos da linguagem	Contexto e ambiente (Cap. 1); fundamentos da sintaxe (Cap. 2); estruturas de dados nativas (Cap. 3).	10 h
II – Dados e objetos	Persistência, expressões regulares e tratamento de erros (Cap. 4); programação orientada a objetos (Cap. 5).	10 h
III – Aplicações e cálculo	Automação e interface gráfica com Tkinter (Cap. 6); computação numérica com NumPy (Cap. 7).	10 h
IV – Análise e projeto	Análise e visualização com pandas, Matplotlib e Seaborn (Cap. 8); projeto final (Cap. 9).	10 h
Carga horária total		40 h

O próximo capítulo dá o primeiro passo concreto: os fundamentos da programação em Python. A partir dele, todo conceito virá acompanhado de código que se executa – de preferência, com você ao teclado.

Resumo do capítulo

- A engenharia de defesa serve a um ecossistema articulado: as **Forças Armadas** (usuárias e demandantes de tecnologia), a **Base Industrial de Defesa** (que projeta e produz), e a **Política e a Estratégia Nacional de Defesa** (que orientam o setor e valorizam a autonomia tecnológica).
- As **tecnologias estratégicas** – IA, ciência de dados, simulação, sistemas autônomos – materializam-se todas em software. Por isso, a programação é a **fundação computacional** do curso.
- Programar, aqui, é uma **competência instrumental**: o objetivo é a autonomia computacional, não a formação de engenheiros de software.
- O Python foi escolhido por sua **legibilidade**, por seu **ecossistema científico maduro**, por ser **livre e multiplataforma** e por ser a mesma linguagem usada em IA e *machine learning*. Adotamos a versão **3.12** como referência.
- O **Google Colab** é o ambiente principal – roda no navegador, sem instalação. O ambiente **local** (Anaconda/`venv` + VS Code) é a alternativa para quando os dados precisam ficar na máquina.
- O **miniprojeto integrador** – um pequeno sistema de apoio à decisão – é construído módulo a módulo e é a base do projeto final do curso.

Armadilhas comuns

- **Achar que “ler” substitui “executar”**. Programação se aprende fazendo. Abrir o *notebook* do capítulo e executar cada exemplo, no seu próprio ritmo, vale mais do que reler o texto.
- **Pular a preparação do ambiente**. Sem um ambiente funcionando, o primeiro encontro se perde com configuração. Deixe o Colab pronto e teste o programa da Listagem 1.1 *antes* do primeiro encontro.

- **Confundir separadores numéricos.** O separador decimal é o ponto (0.007); a vírgula separa valores distintos. Para milhares, use o sublinhado (3_600_000).
- **Tratar a programação como um fim, e não como meio.** O objetivo é resolver problemas de defesa. Mantenha sempre à vista a pergunta “onde isto me seria útil no trabalho?”.

Exercícios

Essencial — fixação, com solução guiada no repositório

Exercício 1.1 Crie uma conta no Google Colab, abra um novo *notebook* e execute o programa da Listagem 1.1. Em seguida, modifique a mensagem para que ela inclua o seu nome e a sua área de atuação (por exemplo, Marinha, Exército, Força Aérea ou Base Industrial de Defesa).

Exercício 1.2 Em uma nova célula, escreva um comentário (linha iniciada por #) explicando, com suas palavras, o que faz a função `print`. Execute a célula e observe que o comentário não produz saída.

Exercício 1.3 Adapte a conta da Amazônia Azul para responder a uma nova pergunta: quantos campos de futebol caberiam em uma área de vigilância de 50 000 km²? Altere apenas o valor da área e execute novamente.

Tático — aplicação a um problema semelhante

Exercício 1.4 Em uma célula de texto (*Markdown*) do seu *notebook*, escreva um parágrafo curto descrevendo um problema do seu cotidiano profissional que você acredita que poderia ser resolvido, ou pelo menos facilitado, com programação. Guarde-o: ele pode se tornar a semente do seu Projeto Final.

Exercício 1.5 Pesquise e registre, em uma célula de texto, qual versão do Python o seu Colab está usando. *Dica:* execute, em uma célula de código, o comando `!python --version`. Compare com a versão de referência do livro (3.12) e reflita: por que pequenas diferenças de versão não afetam os exemplos deste capítulo?

Estratégico — extensão criativa

Exercício 1.6 Considerando o ecossistema de defesa descrito na Seção 1.1, proponha – em meia página, em uma célula de texto – um possível tema para o miniprojeto integrador ambientado na *sua* realidade profissional. Descreva: (a) que dados o sistema gerenciaria; (b) que decisão ele ajudaria a apoiar; (c) quem seria o usuário. Não é preciso escrever código ainda; o objetivo é exercitar a tradução de um problema de defesa em um problema computacional.

Exercício 1.7 Reflita sobre a afirmação “dominar o código é uma forma de nacionalizar capacidade”, feita na Seção 1.1. Em um parágrafo, argumente a favor ou contra essa ideia, trazendo um exemplo concreto – real ou hipotético – do setor de defesa em que a dependência de software de terceiros representaria um risco estratégico.

Fundamentos da Programação em Python

Objetivos do capítulo

Ao final deste capítulo, você será capaz de:

- declarar e usar **variáveis** e reconhecer os **tipos de dados** fundamentais do Python – inteiros, reais, cadeias de caracteres e valores lógicos;
- aplicar os **operadores** aritméticos, de comparação e lógicos para construir expressões;
- ler dados do usuário e apresentar resultados de forma legível, usando `input`, `print` e *f-strings*;
- controlar o fluxo de um programa com **estruturas condicionais** (`if/elif/else`);
- repetir tarefas com **estruturas de repetição** (`for` e `while`) e processar coleções de leituras;
- organizar trechos reutilizáveis em **funções**, aplicando esses fundamentos a problemas reais de engenharia de defesa.

No capítulo anterior, situamos a programação no ecossistema da defesa e preparamos o ambiente. Escrevemos, inclusive, duas pequenas peças de código – mas sem nos determos em *como* elas funcionam. Chegou a

hora de fazer isso. Este é o capítulo em que a sintaxe deixa de ser detalhe e passa a ser ferramenta.

A boa notícia é que os fundamentos do Python são poucos e coerentes. Com variáveis, alguns tipos de dados, operadores e duas formas de controlar o fluxo – decidir e repetir –, já se resolve uma quantidade surpreendente de problemas. Tudo o que vem depois no livro é, em essência, a combinação desses elementos básicos. Por isso, vale percorrer este capítulo com calma e, sobretudo, com o teclado à mão: cada exemplo foi pensado para ser executado e modificado.

2.1 Variáveis e tipos de dados

Um programa, no fundo, manipula *dados*: uma distância, uma velocidade, o nome de um sensor, a indicação de que um alarme está ou não ativo. Para trabalhar com um dado, precisamos guardá-lo em algum lugar com um nome – e esse lugar é a *variável*.

Criar uma variável em Python é direto: escolhe-se um nome, usa-se o sinal de igual e informa-se o valor.

Listagem 2.1: Atribuição de variáveis.

```
1 velocidade_no = 18           # velocidade de um navio, em nós
2 nome_sensor = "Radar-A1"    # identificação de um sensor
3 alvo_detectado = True       # houve detecção?
4 latitude = -22.9            # coordenada, em graus
```

Repare que não foi preciso declarar, de antemão, “que tipo de coisa” cada variável armazena. O Python descobre isso sozinho, a partir do valor atribuído – um traço da linguagem chamado *tipagem dinâmica*. A variável `velocidade_no` passou a guardar um número inteiro; `latitude`, um número real; `nome_sensor`, um texto; e `alvo_detectado`, um valor lógico.

2.1.1 Os quatro tipos fundamentais

Por ora, quatro tipos de dados dão conta de quase tudo:

int – inteiros

Números sem casas decimais: 18, -3, 0, 3_600_000. Úteis para contagens, identificadores, quantidades de meios.

float – reais (ponto flutuante)

Números com casas decimais: 0.007, -22.9, 1852.0. O separador decimal é sempre o *ponto*. Úteis para medidas físicas – distâncias, velocidades, coordenadas.

str – cadeias de caracteres (texto)

Sequências de caracteres entre aspas, simples ou duplas: "Radar-A1", 'Marinha'. Úteis para nomes, rótulos, mensagens.

bool – valores lógicos

Apenas dois valores possíveis: `True` (verdadeiro) e `False` (falso). São a base de toda decisão – um alarme está ativo ou não, um valor está dentro do limite ou não.

A função embutida `type` revela o tipo de qualquer valor – recurso valioso quando um programa não se comporta como esperado:

Listagem 2.2: Descobrindo o tipo de um dado.

```
1 print(type(velocidade_no))    # <class 'int'>
2 print(type(latitude))        # <class 'float'>
3 print(type(nome_sensor))     # <class 'str'>
4 print(type(alvo_detectado))  # <class 'bool'>
```

Nota

O tipo de uma variável pode mudar ao longo do programa, já que é o *valor* que carrega o tipo, não o nome. Isso dá flexibilidade, mas também exige atenção: atribuir um texto a uma variável que antes guardava um número costuma ser fonte de erros sutis. Uma boa prática é manter cada variável fiel a um único papel.

2.1.2 Nomes de variáveis: regras e bom senso

O Python impõe poucas regras para nomes: devem começar por letra ou sublinhado, podem conter letras, números e sublinhados, e não podem coincidir com palavras reservadas da linguagem (como `if`, `for`, `True`). Além das regras, há o bom senso – e, em engenharia, ele importa tanto quanto a sintaxe.

Boa prática

Dê às variáveis nomes que digam *o que elas significam*. Entre `x = 18` e `velocidade_no = 18`, a segunda forma dispensa explicação. A convenção em Python é escrever nomes em minúsculas, separando palavras com sublinhado (`distancia_alvo_km`). Em código que pode ser auditado – como costuma ser o de defesa –, um nome claro é a primeira linha de documentação.

2.2 Operadores

De posse de variáveis, queremos *combiná-las* – somar distâncias, comparar velocidades, verificar condições. É o papel dos operadores.

2.2.1 Operadores aritméticos

Os operadores aritméticos do Python são os que se esperaria, com duas adições úteis:

Operador	Operação	Exemplo
+	soma	10 + 3 resulta em 13
-	subtração	10 - 3 resulta em 7
*	multiplicação	10 * 3 resulta em 30
/	divisão (real)	10 / 3 resulta em 3.333...
//	divisão inteira	10 // 3 resulta em 3
%	resto (módulo)	10 % 3 resulta em 1
**	potência	10 ** 3 resulta em 1000

Vamos a um exemplo com sabor de defesa. A velocidade de navios e aeronaves costuma ser dada em *nós* (milhas náuticas por hora). Um nó equivale a 1,852 km/h. Convertamos a velocidade de um navio de nós para km/h:

Listagem 2.3: Conversão de unidades: nós para km/h.

```
1 velocidade_no = 18           # velocidade em nós
2 fator = 1.852                # 1 nó = 1,852 km/h
3
4 velocidade_kmh = velocidade_no * fator
5 print("Velocidade:", velocidade_kmh, "km/h")

1 Velocidade: 33.336 km/h
```

A ordem das operações segue a convenção matemática usual (potências antes de multiplicações e divisões, que vêm antes de somas e subtrações), e os parênteses podem – e devem – ser usados para deixar a intenção explícita.

Armadilha comum

Cuidado com a diferença entre `/` e `//`. A divisão `/` sempre produz um `float`, mesmo quando o resultado é exato: `10 / 2` resulta em `5.0`, não `5`. Já `//` descarta a parte fracionária. Trocar uma pela outra é um erro silencioso clássico – o programa não acusa falha, apenas entrega um número ligeiramente errado, o que em um cálculo de engenharia pode passar despercebido até tarde demais.

2.2.2 Operadores de comparação

Comparações produzem sempre um valor lógico (`bool`) – são a matéria-prima das decisões que veremos adiante:

Operador	Significado	Exemplo (resultado)
==	igual a	18 == 18 → True
!=	diferente de	18 != 20 → True
>	maior que	18 > 20 → False
<	menor que	18 < 20 → True
>=	maior ou igual a	18 >= 18 → True
<=	menor ou igual a	18 <= 17 → False

Armadilha comum

Não confunda = com ==. Um único sinal de igual *atribui* um valor a uma variável; o sinal duplo *compara* dois valores. Escrever `if velocidade = 18` é um erro de sintaxe – o correto, para comparar, é `if velocidade == 18`.

2.2.3 Operadores lógicos

Para combinar condições, o Python usa palavras, e não símbolos – mais um traço de sua legibilidade: `and` (e), `or` (ou) e `not` (não).

Listagem 2.4: Combinando condições com operadores lógicos.

```

1 velocidade_kmh = 33.3
2 em_area_restrita = True
3
4 # A condição só é verdadeira se ambas as partes forem
   verdadeiras
5 alerta = (velocidade_kmh > 30) and em_area_restrita
6 print("Disparar alerta?", alerta)    # True

```

O resultado de `(velocidade_kmh > 30)` é `True`; combinado com `em_area_restrita` (também `True`) pelo `and`, produz `True`. Bastaria uma das partes ser falsa para o `and` resultar em `False`.

2.3 Entrada e saída de dados

Programas úteis raramente trabalham apenas com valores fixos no código. Eles *recebem* dados – de um usuário, de um arquivo, de um sensor – e *apresentam* resultados. Vimos a saída com `print`; vejamos agora a entrada, e uma forma mais elegante de formatar a saída.

2.3.1 Lendo dados com `input`

A função `input` pausa o programa, exibe uma mensagem e aguarda que o usuário digite algo. O que for digitado é devolvido sempre como `str`:

Listagem 2.5: Lendo um valor do usuário.

```
1 texto = input("Informe a velocidade em nós: ")
2 velocidade_no = float(texto)          # converte o texto em
    número real
3 velocidade_kmh = velocidade_no * 1.852
4 print("Equivale a", velocidade_kmh, "km/h")
```

A conversão na segunda linha é essencial: como `input` entrega texto, tentar multiplicar `texto` por `1.852` diretamente provocaria erro – ou, no caso da soma, concatenaria textos em vez de somar números. As funções `int`, `float` e `str` convertem valores de um tipo para outro; esse *cast* é uma operação corriqueira e importante.

Armadilha comum

Tudo o que vem de `input` é texto. Se você espera um número e não converte, o Python pode não acusar erro de imediato – `"2" + "3"` resulta em `"23"`, não `5`. Sempre converta a entrada para o tipo esperado, e o quanto antes.

2.3.2 Saída formatada com *f-strings*

Concatenar texto e números com vírgulas em `print` funciona, mas fica desajeitado. A partir do Python 3.6, dispomos das *f-strings* – cadeias prefixadas pela letra `f`, dentro das quais qualquer expressão entre chaves é

avaliada e inserida no texto:

Listagem 2.6: Saída formatada com f-strings.

```
1 velocidade_no = 18
2 velocidade_kmh = velocidade_no * 1.852
3
4 print(f"O navio a {velocidade_no} nós navega a
   {velocidade_kmh:.1f} km/h.")
```

```
1 O navio a 18 nós navega a 33.3 km/h.
```

O trecho `:.1f` dentro das chaves é um *especificador de formato*: pede que o número seja exibido como real com uma casa decimal. As *f-strings* tornam o código mais legível e serão usadas ao longo de todo o livro.

2.4 Estruturas condicionais

Até aqui, nossos programas executavam todas as linhas, de cima a baixo, sem exceção. Mas decidir é da essência de qualquer sistema de apoio à decisão: *se* a velocidade exceder um limite, *então* emitir um alerta. É o que fazem as estruturas condicionais.

2.4.1 `if, elif e else`

A forma básica usa a palavra `if` seguida de uma condição e de dois-pontos; o bloco que depende da condição vem *indentado* (deslocado à direita):

Listagem 2.7: Decisão simples com `if/else`.

```
1 velocidade_kmh = 42.0
2 limite_kmh = 40.0
3
4 if velocidade_kmh > limite_kmh:
5     print("ALERTA: velocidade acima do limite.")
6 else:
7     print("Velocidade dentro do limite.")
```

```
1 ALERTA: velocidade acima do limite.
```


Quando há mais de duas possibilidades, encadeiam-se condições com `elif` (contração de *else if*). Suponha que se queira classificar uma detecção por faixa de velocidade:

Listagem 2.8: Classificação em faixas com `elif`.

```

1 velocidade_kmh = 75.0
2
3 if velocidade_kmh < 30:
4     classe = "lento"
5 elif velocidade_kmh < 60:
6     classe = "moderado"
7 elif velocidade_kmh < 100:
8     classe = "rápido"
9 else:
10    classe = "muito rápido"
11
12 print(f"Contato classificado como: {classe}.")

```

```

1 Contato classificado como: rápido.

```

As condições são avaliadas de cima para baixo; assim que uma resulta em `True`, seu bloco é executado e as demais são ignoradas. O `else` final é opcional e funciona como “nenhuma das anteriores”.

Contexto de defesa

A lógica condicional é o coração de um sistema de apoio à decisão. Regras como “se o contato está em área restrita *e* acima da velocidade permitida, então elevar o nível de alerta” são exatamente o que se traduz em `if/elif/else`. A clareza dessas regras no código não é só elegância: em um sistema auditável, é o que permite explicar, depois, *por que* uma decisão foi recomendada.

Armadilha comum

Em Python, a **indentação não é decoração – é sintaxe**. O bloco que pertence a um `if` é definido pelo deslocamento à direita (em geral, quatro espaços). Misturar espaços e tabulações, ou alinhar de forma inconsistente, provoca o erro `IndentationError`. Configure

o editor para inserir quatro espaços ao pressionar *Tab* e o problema desaparece.

2.5 Estruturas de repetição

A maior vantagem do computador sobre o cálculo manual é a repetição incansável. Processar mil leituras de um sensor é, para o Python, tão simples quanto processar uma – desde que saibamos pedir a repetição. Há duas formas: o `for` e o `while`.

2.5.1 O laço `for`

O `for` percorre os elementos de uma coleção, um a um. Imagine uma série de leituras de velocidade vindas de um sensor de telemetria, guardadas em uma lista (estrutura que estudaremos a fundo no Capítulo 3; por ora, basta vê-la como uma sequência de valores entre colchetes):

Listagem 2.9: Percorrendo leituras de telemetria com `for`.

```
1 # Leituras de velocidade (km/h) recebidas de um sensor
2 leituras = [28.4, 33.1, 41.7, 39.0, 45.2]
3 limite = 40.0
4
5 for leitura in leituras:
6     if leitura > limite:
7         print(f"Leitura {leitura} km/h: ACIMA do limite.")
8     else:
9         print(f"Leitura {leitura} km/h: normal.")
```

```
1 Leitura 28.4 km/h: normal.
2 Leitura 33.1 km/h: normal.
3 Leitura 41.7 km/h: ACIMA do limite.
4 Leitura 39.0 km/h: normal.
5 Leitura 45.2 km/h: ACIMA do limite.
```

A cada volta do laço, a variável `leitura` assume o próximo valor da lista, e o bloco indentado é executado. Note como condicional e repetição se combinam naturalmente – é dessa combinação que nasce a maior parte

da lógica útil.

Quando se precisa repetir um número fixo de vezes, em vez de percorrer uma coleção, usa-se `range`, que gera uma sequência de números inteiros:

Listagem 2.10: Repetição contada com `range`.

```
1 # Tabela de conversão: 1 a 5 nós em km/h
2 for nos in range(1, 6):          # gera 1, 2, 3, 4, 5
3     kmh = nos * 1.852
4     print(f"{nos} nó(s) = {kmh:.3f} km/h")
```

Nota

O `range(1, 6)` gera os números de 1 a 5 – o limite final é *exclusivo*. Essa convenção, comum em Python, surpreende no início, mas tem suas vantagens; por ora, basta lembrar: o último valor pedido não entra. Já `range(5)`, com um único argumento, gera de 0 a 4.

2.5.2 O laço `while`

O `while` repete um bloco *enquanto* uma condição permanecer verdadeira. É a escolha natural quando não se sabe de antemão quantas repetições serão necessárias – por exemplo, acumular leituras até que um total seja atingido:

Listagem 2.11: Repetição condicional com `while`.

```
1 contador = 0
2 acumulado = 0.0
3
4 # Soma leituras simuladas até ultrapassar 100 km/h acumulados
5 while acumulado < 100:
6     acumulado = acumulado + 25.0    # cada leitura adiciona
7     contador = contador + 1         25 km/h
8
9 print(f"Foram necessárias {contador} leituras para
    ultrapassar 100.")
```

Armadilha comum

Todo `while` precisa, em algum momento, tornar sua condição falsa – caso contrário, o programa entra em *laço infinito* e nunca termina. No exemplo acima, é a linha que incrementa `acumulado` que garante o fim. Esquecer de atualizar a variável da condição é um dos erros mais comuns – e, no Colab, exige interromper a execução manualmente.

2.6 Funções: organizando e reutilizando código

Já usamos funções prontas – `print`, `input`, `type`, `round`. Mas o Python permite que *definamos as nossas*. Uma função é um trecho de código com nome, que recebe dados de entrada (os *parâmetros*), faz algo e, em geral, devolve um resultado. Funções evitam repetição e dão nome às intenções – dois pilares do código manutenível.

Voltemos à conversão de nós para km/h, que já fizemos algumas vezes. Em vez de repetir o cálculo, vamos encapsulá-lo:

Listagem 2.12: Definindo uma função de conversão.

```

1 def nos_para_kmh(velocidade_no):
2     """Converte uma velocidade de nós para km/h."""
3     return velocidade_no * 1.852
4
5     # Agora a conversão é uma chamada, reutilizável quantas
6     vezes quisermos
7
8 print(nos_para_kmh(18))      # 33.336
9 print(nos_para_kmh(25))      # 46.3
10 print(nos_para_kmh(10))     # 18.52

```

A palavra `def` inicia a definição; segue-se o nome, os parâmetros entre parênteses e dois-pontos. O bloco indentado é o corpo da função. A linha entre aspas triplas logo abaixo do cabeçalho é a *docstring* – uma descrição do que a função faz, que serve de documentação. A palavra `return` indica o valor devolvido por quem chamou a função.

Funções podem receber mais de um parâmetro. Um exemplo recorrente em engenharia é o cálculo de um *alcance* simplificado: a distância horizontal percorrida por um projétil lançado a uma certa velocidade e ângulo,

desprezando a resistência do ar. A fórmula é $R = \frac{v^2 \sin(2\theta)}{g}$.

Listagem 2.13: Função com vários parâmetros: alcance balístico simplificado.

```
1 import math      # biblioteca padrão de funções matemáticas
2
3 def alcance(velocidade_ms, angulo_graus, g=9.81):
4     """Alcance horizontal (m) de um projétil, sem
5     resistência do ar."""
6     angulo_rad = math.radians(angulo_graus)
7     return (velocidade_ms ** 2) * math.sin(2 * angulo_rad) /
8         g
9
10 # Velocidade de 50 m/s, ângulo de 45 graus
11 print(f"Alcance: {alcance(50, 45):.1f} m")
```

```
1 Alcance: 254.8 m
```

Dois detalhes valem nota. O comando `import math` torna disponível a biblioteca matemática padrão, de onde vêm `radians` e `sin` – o Python traz “baterias inclusas”, um vasto conjunto de funções prontas. E o parâmetro `g=9.81` tem um *valor padrão*: se quem chama não o informar, adota-se 9,81 m/s²; mas seria possível passar outro valor, por exemplo para simular outro corpo celeste.

Boa prática

Uma função deve fazer *uma* coisa e fazê-la bem – e seu nome deve dizer qual. Se você se pega explicando o que uma função faz com a palavra “e” (“ela converte *e* também imprime *e* ainda grava”), provavelmente são três funções. Funções pequenas e bem-nomeadas são mais fáceis de testar, reaproveitar e auditar – qualidades especialmente valiosas em sistemas de defesa.

2.7 Juntando tudo: um pequeno processador de telemetria

Os fundamentos deste capítulo já bastam para escrever algo genuinamente útil. Vamos combinar variáveis, operadores, uma função, um laço e uma

condicional em um único programa que processa um conjunto de leituras de telemetria e produz um pequeno relatório – um embrião do tipo de lógica que sustentará o miniprojeto.

Listagem 2.14: Processador de telemetria reunindo os fundamentos.

```

1 def classificar(velocidade_kmh, limite):
2     """Devolve 'alerta' se acima do limite, senão
3     'normal'."""
4     if velocidade_kmh > limite:
5         return "alerta"
6     return "normal"
7
8 leituras_no = [12, 18, 24, 9, 27]    # leituras em nós
9 limite_kmh = 40.0
10 total_alertas = 0
11
12 print("Relatório de telemetria")
13 print("-" * 30)
14
15 for leitura_no in leituras_no:
16     kmh = nos_para_kmh(leitura_no)    # reutiliza a
17     estado = classificar(kmh, limite_kmh)
18     if estado == "alerta":
19         total_alertas = total_alertas + 1
20         print(f"{leitura_no:>3} nós -> {kmh:6.1f} km/h
21         [{estado}]")
22
23 print("-" * 30)
24 print(f"Total de leituras em alerta: {total_alertas}")

```

```

1 Relatório de telemetria
2 -----
3 12 nós -> 22.2 km/h [normal]
4 18 nós -> 33.3 km/h [normal]
5 24 nós -> 44.4 km/h [alerta]
6 9 nós -> 16.7 km/h [normal]
7 27 nós -> 50.0 km/h [alerta]
8 -----
9 Total de leituras em alerta: 2

```

Em pouco mais de vinte linhas, o programa converte unidades, classifica cada leitura, conta as ocorrências de alerta e apresenta o resultado de

forma legível. Note os recursos de formatação nas *f-strings*: `>3` alinha o número à direita em três colunas, e `>6.1f` reserva seis colunas e uma casa decimal – detalhes que transformam uma saída desalinhada em um relatório apresentável. Cada peça aqui veio de uma seção deste capítulo. É essa combinação de elementos simples que, ampliada, dará origem ao sistema de apoio à decisão do miniprojeto.

2.8 O caminho à frente

Este capítulo entregou o vocabulário básico da linguagem: variáveis e tipos, operadores, entrada e saída, decisão, repetição e funções. São os tijolos com que se constrói tudo o mais. No próximo capítulo, voltamos a atenção para as **estruturas de dados nativas** do Python – listas, tuplas, dicionários e conjuntos –, que nos permitirão organizar não apenas *uma* leitura, mas coleções inteiras de informações de forma rica e expressiva. E será ali, ao modelar os dados do sistema, que daremos a primeira machadada concreta no miniprojeto integrador.

Resumo do capítulo

- As **variáveis** guardam dados sob um nome; o Python usa **tipagem dinâmica** – o valor define o tipo. Os quatro tipos fundamentais são `int`, `float`, `str` e `bool`.
- Os **operadores** aritméticos incluem `//` (divisão inteira) e `%` (resto); os de **comparação** produzem valores lógicos; e os **lógicos** são `and`, `or` e `not`.
- `input` lê dados sempre como **texto** – converta com `int` ou `float`. As **f-strings** (`f"..."`) formatam a saída de modo legível, com especificadores como `>6.1f`.
- As **estruturas condicionais** (`if/elif/else`) dão ao programa a capacidade de decidir; são o coração de um sistema de apoio à decisão.

- As **estruturas de repetição** – `for` (percorre coleções ou `range`) e `while` (repete enquanto a condição for verdadeira) – automatizam tarefas repetitivas, como processar leituras de telemetria.
- As **funções** (`def ... return`) encapsulam trechos reutilizáveis, com parâmetros e valores padrão, tornando o código mais limpo, testável e auditável.

Armadilhas comuns

- **Confundir = com ==.** O primeiro atribui; o segundo compara. Em uma condição, é sempre `==`.
- **Esquecer de converter a entrada.** Tudo o que vem de `input` é texto. Sem `int` ou `float`, somas viram concatenações e comparações se comportam de forma inesperada.
- **Trocar / por //.** A divisão real e a inteira produzem resultados diferentes; o erro é silencioso e perigoso em cálculos de engenharia.
- **Errar a indentação.** Em Python, o recuo define os blocos. Use sempre quatro espaços e seja consistente, ou surgirá o `IndentationError`.
- **Criar um laço `while` infinito.** Garanta que a condição se tornará falsa em algum momento, atualizando a variável que a controla.
- **Confiar nos limites do `range`.** `range(1, 6)` vai de 1 a 5: o limite final é exclusivo.

Exercícios

Essencial — fixação, com solução guiada no repositório

Exercício 2.1 Crie três variáveis: o nome de um sensor (texto), o número de leituras que ele registrou (inteiro) e a sua leitura média (real). Em seguida, use uma *f-string* para imprimir uma frase que combine os três valores, exibindo a média com duas casas decimais.

Exercício 2.2 Escreva um programa que leia, com `input`, uma distância em milhas náuticas e a converta para quilômetros (1 milha náutica = 1,852 km). Lembre-se de converter a entrada para `float` antes de calcular.

Exercício 2.3 Dada a leitura de velocidade de um contato, escreva um `if/else` que imprima “acima do limite” se a velocidade ultrapassar 50 km/h, e “dentro do limite” caso contrário. Teste com diferentes valores.

Tático — aplicação a um problema semelhante

Exercício 2.4 Reescreva a classificação em faixas da Listagem 2.8 como uma **função** `classificar_velocidade(velocidade_kmh)` que devolva a classe correspondente. Chame-a para três velocidades diferentes e imprima os resultados.

Exercício 2.5 Dada a lista `leituras = [22, 35, 48, 19, 51, 40]` (em km/h), escreva um laço `for` que conte quantas leituras estão acima de 40 km/h e, ao final, imprima esse total. (Não use bibliotecas externas – apenas os fundamentos deste capítulo.)

Exercício 2.6 Escreva uma função `converter_temperatura(celsius)` que converta graus Celsius para Fahrenheit ($F = C \times 9/5 + 32$) e a use, com um laço `for` sobre `range`, para imprimir uma tabela de 0 a 40 °C, de 10 em 10 graus.

Estratégico — extensão criativa

Exercício 2.7 Amplie o processador de telemetria da Listagem 2.14 para que, além de contar os alertas, ele também calcule e exiba a *maior* velocidade registrada (em km/h) e a *média* das leituras. *Dica:* use variáveis acumuladoras, atualizadas dentro do laço.

Exercício 2.8 Escreva um pequeno programa interativo que, em um laço `while`, leia repetidamente leituras de velocidade digitadas pelo usuário (em nós), as converta para km/h e as classifique, encerrando quando o usuário digitar a palavra `fim`. Ao final, informe quantas leituras foram processadas. *Dica:* verifique se o texto digitado é “fim” antes de tentar convertê-lo para número.

Estruturas de Dados Nativas

◎ Objetivos do capítulo

Ao final deste capítulo, você será capaz de:

- criar e manipular **listas** – acessar elementos por índice, fatiá-las e usar seus principais métodos;
- escrever **compreensões de lista** para construir e filtrar coleções de forma concisa;
- distinguir **tuplas** de listas e empregar a imutabilidade e o desempacotamento a seu favor;
- organizar dados em **dicionários** (pares chave–valor) e percorrê-los;
- usar **conjuntos** para garantir unicidade e realizar operações de conjunto;
- escolher a **estrutura adequada** a cada problema de engenharia de defesa;
- iniciar o **miniprojeto integrador**, modelando em memória os dados de um sistema de apoio à decisão.

No capítulo anterior, trabalhamos com valores isolados: uma velocidade, um nome, um sinalizador lógico. Mas os problemas reais raramente envolvem um único dado. Um sensor produz *muitas* leituras; uma área de vigilância acumula *muitas* ocorrências; uma frota tem *muitos* meios.

Precisamos de formas de agrupar dados – e o Python oferece quatro estruturas nativas, cada uma talhada para um tipo de situação: **listas**, **tuplas**, **dicionários** e **conjuntos**.

Dominar essas quatro estruturas – e, sobretudo, saber *quando* usar cada uma – é o que separa um código que apenas funciona de um código que é claro, eficiente e fácil de manter. É também o primeiro passo concreto do nosso miniprojeto: ao final do capítulo, teremos modelado, em memória, os dados do sistema de apoio à decisão que construiremos ao longo do livro.

3.1 Listas: a estrutura de trabalho

A *lista* é a estrutura mais versátil e a mais usada do Python. Ela guarda uma sequência *ordenada* de elementos, que podem ser de qualquer tipo, e é *mutável* – isto é, pode ser alterada depois de criada. Listas são escritas entre colchetes, com os elementos separados por vírgula.

Listagem 3.1: Criando listas.

```

1  # Um inventário de meios (embarcações de uma flotilha)
2  meios = ["Fragata", "Corveta", "Navio-Patrolha", "Rebocador"]
3
4  # Leituras de velocidade de um sensor (km/h)
5  leituras = [28.4, 33.1, 41.7, 39.0, 45.2]
6
7  # Uma lista pode até misturar tipos, embora raramente
   convenha
8  registro = ["Radar-A1", 3, True]
9
10 print(meios)
11 print("Quantidade de meios:", len(meios))

```

```

1  ['Fragata', 'Corveta', 'Navio-Patrolha', 'Rebocador']
2  Quantidade de meios: 4

```

A função `len` devolve o número de elementos – uma das operações mais frequentes ao trabalhar com qualquer coleção.

3.1.1 Acessando elementos por índice

Cada elemento de uma lista tem uma posição, o *índice*, que começa em **zero**. O primeiro elemento está no índice 0, o segundo no índice 1, e assim por diante. Índices negativos contam a partir do fim: -1 é o último elemento.

Listagem 3.2: Acesso por índice.

```
1 meios = ["Fragata", "Corveta", "Navio-Patrolha", "Rebocador"]
2
3 print(meios[0])      # Fragata      (primeiro)
4 print(meios[2])      # Navio-Patrolha
5 print(meios[-1])     # Rebocador   (último)
6 print(meios[-2])     # Navio-Patrolha (penúltimo)
```

Armadilha comum

A contagem de índices começa em **zero**, não em um. Em uma lista de quatro elementos, os índices válidos vão de 0 a 3 – e de -1 a -4. Tentar acessar `meios[4]` provoca o erro `IndexError: list index out of range`. Esse deslize – o “erro de um a mais” – é talvez o mais clássico da programação; desconfie sempre dos limites.

3.1.2 Fatiamento: recortando sublistas

Além de acessar um elemento, podemos extrair uma *fatia* (*slice*) da lista, usando a notação `[início:fim]`. Como no `range` do capítulo anterior, o índice final é *exclusivo*.

Listagem 3.3: Fatiamento de listas.

```
1 leituras = [28.4, 33.1, 41.7, 39.0, 45.2]
2
3 print(leituras[0:3]) # [28.4, 33.1, 41.7] (índices 0, 1 e
4                    2)
5 print(leituras[2:]) # [41.7, 39.0, 45.2] (do índice 2 ao
6                    fim)
7 print(leituras[:2]) # [28.4, 33.1]      (do início ao í
8                    ndice 1)
9 print(leituras[-2:]) # [39.0, 45.2]     (as duas ú
10                   ltimas)
```

O fatiamento é uma ferramenta poderosa para trabalhar com janelas de dados – por exemplo, as últimas N leituras de um sensor, ou um intervalo específico de um registro temporal.

3.1.3 Modificando listas

Por serem mutáveis, as listas oferecem um rico conjunto de *métodos* – funções acopladas ao objeto, chamadas com a notação de ponto. Os mais usados:

Método	O que faz
<code>lista.append(x)</code>	adiciona x ao final
<code>lista.insert(i, x)</code>	insere x na posição i
<code>lista.remove(x)</code>	remove a primeira ocorrência de x
<code>lista.pop(i)</code>	remove e devolve o elemento da posição i
<code>lista.sort()</code>	ordena a lista <i>no lugar</i>
<code>lista.reverse()</code>	inverte a ordem dos elementos
<code>lista.count(x)</code>	conta quantas vezes x aparece

Listagem 3.4: Modificando uma lista de ocorrências.

```

1  ocorrencias = ["contato-norte", "contato-leste"]
2
3  ocorrencias.append("contato-sul")           # adiciona ao final
4  ocorrencias.insert(0, "contato-urgente")    # insere no início
5  print(ocorrencias)
6
7  ocorrencias.remove("contato-leste")         # remove pelo valor
8  ultima = ocorrencias.pop()                 # remove e devolve
                                           a última
9  print("Removida:", ultima)
10 print("Restantes:", ocorrencias)

```

```

1  ['contato-urgente', 'contato-norte', 'contato-leste',
   'contato-sul']
2  Removida: contato-sul
3  Restantes: ['contato-urgente', 'contato-norte']

```

Armadilha comum

Métodos como `sort` e `reverse` alteram a lista *no lugar* e devolvem `None`. Escrever `leituras = leituras.sort()` é um erro comum: a variável passa a valer `None`, e os dados se perdem. O certo é apenas `leituras.sort()`. Se você precisa de uma *cópia* ordenada sem alterar a original, use a função `sorted(leituras)`.

3.1.4 Percorrendo listas

Já vimos, no Capítulo 2, como percorrer uma lista com `for`. Quando se precisa do índice *e* do valor ao mesmo tempo, a função `enumerate` é a forma idiomática:

Listagem 3.5: Percorrendo com índice usando `enumerate`.

```
1 leituras = [28.4, 33.1, 41.7, 39.0, 45.2]
2
3 for indice, valor in enumerate(leituras):
4     print(f"Leitura {indice}: {valor} km/h")
```

3.2 Compreensão de listas

Uma tarefa muito comum é construir uma nova lista a partir de outra – convertendo, filtrando ou transformando seus elementos. Poderíamos fazê-lo com um laço e `append`, mas o Python oferece uma forma mais concisa e elegante: a *compreensão de lista* (*list comprehension*).

Compare as duas formas de converter uma lista de velocidades de nós para km/h:

Listagem 3.6: Do laço à compreensão de lista.

```
1 leituras_no = [12, 18, 24, 9, 27]
2
3 # Forma tradicional, com laço
4 kmh = []
5 for no in leituras_no:
6     kmh.append(no * 1.852)
7
```

```

8 # Forma idiomática, com compreensão de lista
9 kmh = [no * 1.852 for no in leituras_no]
10
11 print(kmh)

```

As duas produzem o mesmo resultado, mas a segunda diz, em uma linha, exatamente o que pretende: “para cada `no` em `leituras_no`, calcule `no * 1.852`”. A compreensão também aceita um filtro, com `if`:

Listagem 3.7: Compreensão de lista com filtro.

```

1 leituras = [28.4, 33.1, 41.7, 39.0, 45.2]
2
3 # Apenas as leituras acima de 40 km/h
4 acima = [x for x in leituras if x > 40]
5 print(acima)           # [41.7, 45.2]
6
7 # Quantas estão acima do limite?
8 print(len(acima))     # 2

```

Boa prática

A compreensão de lista é mais legível que o laço equivalente *quando a lógica é simples* – uma transformação, um filtro, ou ambos. Se a regra começa a exigir vários `if`, cálculos intermediários ou aninhamentos profundos, volte ao laço tradicional: clareza vale mais que concisão. A compreensão é uma ferramenta de elegância, não um teste de esperteza.

3.3 Tuplas: dados que não mudam

A *tupla* é, à primeira vista, uma lista escrita com parênteses em vez de colchetes. A diferença essencial está em uma palavra: tuplas são **imutáveis**. Uma vez criada, uma tupla não pode ter seus elementos alterados, adicionados ou removidos.

Listagem 3.8: Criando e usando tuplas.

```

1 # Uma coordenada geográfica: latitude e longitude
2 posicao = (-22.9, -43.2)

```

```

3
4 print(posicao[0])      # -22.9  (acesso por índice, como na
                        lista)
5 print(posicao[1])      # -43.2
6
7 # Tentar alterar provoca erro:
8 # posicao[0] = -23.0    -> TypeError: 'tuple' object does
                        not support
9 #                                item assignment

```

Por que desejar uma estrutura que *não* se pode alterar? Justamente porque há dados que *não devem* mudar. Uma coordenada, um par de constantes, um registro que precisa permanecer íntegro: a imutabilidade é uma garantia, não uma limitação. Em sistemas auditáveis, poder afirmar que um dado não foi alterado acidentalmente tem valor concreto.

3.3.1 Desempacotamento

Um recurso elegante das tuplas (que também vale para listas) é o *desempacotamento*: atribuir, de uma vez, cada elemento a uma variável.

Listagem 3.9: Desempacotamento de tuplas.

```

1 posicao = (-22.9, -43.2)
2
3 latitude, longitude = posicao      # desempacota nos dois
                        nomes
4 print(f"Lat {latitude}, Lon {longitude}")
5
6 # Útil também para trocar valores sem variável auxiliar
7 a, b = 10, 20
8 a, b = b, a
9 print(a, b)                      # 20 10

```

Nota

Funções que precisam devolver mais de um valor o fazem, em Python, retornando uma tupla. Por exemplo, uma função que calcula mínimo e máximo de uma série pode terminar com `return menor, maior`; quem chama recebe os dois com `m, M`

= estatisticas(leituras). É um padrão que você verá com frequência.

3.4 Dicionários: dados com rótulo

Listas e tuplas guardam dados por *posição*. Mas muitas vezes queremos acessar dados por um *nome* significativo, não por um índice numérico. É o papel do *dicionário*: uma coleção de pares **chave–valor**, escrita entre chaves.

Listagem 3.10: Um dicionário descrevendo uma ocorrência.

```

1  ocorrencia = {
2      "id": 101,
3      "sensor": "Radar-A1",
4      "tipo": "contato de superfície",
5      "velocidade_kmh": 44.4,
6      "em_area_restrita": True,
7  }
8
9  print(ocorrencia["sensor"])      # Radar-A1
10 print(ocorrencia["velocidade_kmh"]) # 44.4

```

Compare a clareza de `ocorrencia["sensor"]` com um hipotético `ocorrencia[1]`: o dicionário torna o código autoexplicativo. As chaves são, em geral, textos; os valores podem ser de qualquer tipo – inclusive listas ou outros dicionários.

3.4.1 Acessando, adicionando e alterando

Listagem 3.11: Manipulando um dicionário.

```

1  ocorrencia = {"id": 101, "sensor": "Radar-A1",
2               "velocidade_kmh": 44.4}
3
4  # Adicionar ou alterar é igual: atribuir a uma chave
5  ocorrencia["classe"] = "rápido"      # nova chave
6  ocorrencia["velocidade_kmh"] = 46.0  # altera valor
7                                     existente

```

```

6
7 # Acesso seguro com .get (evita erro se a chave não existir)
8 print(ocorrencia.get("latitude", "não informada"))
9
10 # Remover uma chave
11 del ocorrencia["id"]
12 print(ocorrencia)

```

Armadilha comum

Acessar uma chave inexistente com colchetes – `ocorrencia["latitude"]` – provoca `KeyError`. Quando não há certeza de que a chave existe, prefira o método `.get(chave, padrão)`, que devolve um valor padrão em vez de interromper o programa. Em dados vindos do mundo real – de sensores, de arquivos, de digitação – nunca presuma que um campo está presente.

3.4.2 Percorrendo dicionários

Para percorrer um dicionário, o método `.items()` entrega, a cada volta, a chave e o valor:

Listagem 3.12: Percorrendo um dicionário com `items()`.

```

1 ocorrencia = {
2     "sensor": "Radar-A1",
3     "tipo": "contato de superfície",
4     "velocidade_kmh": 44.4,
5 }
6
7 for chave, valor in ocorrencia.items():
8     print(f"{chave}: {valor}")

```

```

1 sensor: Radar-A1
2 tipo: contato de superfície
3 velocidade_kmh: 44.4

```

Os métodos `.keys()` e `.values()` entregam, respectivamente, apenas as chaves ou apenas os valores – úteis quando só um dos dois interessa.

3.4.3 O padrão lista de dicionários

Eis a combinação mais importante deste capítulo, e a base do nosso mini-projeto: uma **lista de dicionários**. Cada dicionário descreve um registro – uma ocorrência, um meio, uma leitura – com seus campos rotulados; a lista reúne todos os registros. É a forma natural de representar uma “tabela” de dados em Python.

Listagem 3.13: Uma lista de dicionários: registro de ocorrências.

```

1  ocorrencias = [
2      {"id": 1, "sensor": "Radar-A1", "velocidade_kmh": 22.2,
3       "alerta": False},
4      {"id": 2, "sensor": "Radar-A1", "velocidade_kmh": 44.4,
5       "alerta": True},
6      {"id": 3, "sensor": "Radar-B2", "velocidade_kmh": 50.0,
7       "alerta": True},
8  ]
9
10 # Quantas ocorrências estão em alerta?
11 em_alerta = [o for o in ocorrencias if o["alerta"]]
12 print(f"Ocorrências em alerta: {len(em_alerta)}")
13
14 # Listar os sensores envolvidos em alertas
15 for o in em_alerta:
16     print(f"    - {o['sensor']} a {o['velocidade_kmh']} km/h")

```

```

1  Ocorrências em alerta: 2
2      - Radar-A1 a 44.4 km/h
3      - Radar-B2 a 50.0 km/h

```

Repare como compreensão de lista, dicionário e *f-string* se combinam com naturalidade. Esse pequeno trecho já é, em essência, uma consulta a uma base de dados em memória.

3.5 Conjuntos: unicidade e pertencimento

A última estrutura, o *conjunto* (*set*), guarda elementos **únicos e não ordenados**. Sua especialidade é responder, com eficiência, a duas perguntas: “este elemento já está aqui?” e “quais são os elementos distintos?”. Conjuntos são escritos entre chaves, mas sem o par chave–valor dos dicionários.

Listagem 3.14: Removendo duplicatas com conjuntos.

```

1  # Sensores que registraram ocorrências (com repetições)
2  sensores = ["Radar-A1", "Radar-B2", "Radar-A1", "Radar-A1",
3             "Radar-B2"]
4
5  # Quais sensores distintos atuaram?
6  distintos = set(sensores)
7  print(distintos)           # {'Radar-A1', 'Radar-B2'}
8  print("Sensores distintos:", len(distintos))  # 2
9
10 # Pertencimento é imediato e eficiente
11 print("Radar-A1" in distintos)  # True
12 print("Radar-C3" in distintos)  # False

```

Conjuntos também suportam as operações clássicas da teoria de conjuntos – união (`|`), interseção (`&`) e diferença (`-`) –, úteis para comparar grupos:

Listagem 3.15: Operações de conjunto.

```

1  sensores_manha = {"Radar-A1", "Radar-B2", "Sonar-1"}
2  sensores_tarde = {"Radar-B2", "Sonar-1", "Radar-C3"}
3
4  print(sensores_manha & sensores_tarde)  # ativos nos dois
5                                         turnos
6  print(sensores_manha | sensores_tarde)  # ativos em algum
7                                         turno
8  print(sensores_manha - sensores_tarde)  # só de manhã
9
10 {'Radar-B2', 'Sonar-1'}
11 {'Radar-A1', 'Radar-B2', 'Sonar-1', 'Radar-C3'}
12 {'Radar-A1'}

```

3.6 Escolhendo a estrutura certa

As quatro estruturas não competem entre si – complementam-se. Saber escolher é uma habilidade de engenharia tão importante quanto saber usar. O quadro a seguir resume os critérios de decisão.

Estrutura	Quando usar	Exemplo de defesa
Lista	sequência ordenada que pode mudar	leituras de um sensor, em ordem de chegada
Tupla	dados fixos que não devem mudar	uma coordenada (lat, lon); um par de constantes
Dicionário	dados acessados por um rótulo	uma ocorrência com seus campos nomeados
Conjunto	coleção de itens únicos; teste de pertencimento	sensores distintos que atuaram em uma missão

Contexto de defesa

A escolha da estrutura de dados é uma decisão de modelagem – e modelar bem é traduzir fielmente a realidade para o código. Uma frota é uma *lista* de meios; cada meio é um *dicionário* de atributos; sua posição é uma *tupla* imutável; e os tipos de meio presentes formam um *conjunto*. Um modelo de dados que espelha a realidade torna o restante do sistema quase óbvio de escrever. Um modelo torto contamina tudo o que vem depois.

3.7 Miniprojeto: modelando os dados (Módulo 1)

Chegou o momento prometido: damos início ao **miniprojeto integrador**. Conforme apresentado na Seção 1.5, construiremos, módulo a módulo, um pequeno sistema de apoio à decisão. O tema que adotaremos no livro é um **sistema de registro e análise de ocorrências de monitoramento** – por exemplo, detecções em uma área de vigilância. Você é encorajado a adaptá-lo, desde já, ao problema da sua realidade profissional.

A tarefa do Módulo 1 é *modelar os dados em memória*. Com o que aprendemos neste capítulo, a escolha é natural: cada ocorrência será um **dicionário**;

o conjunto de ocorrências, uma **lista**. Vamos também escrever as primeiras operações sobre esses dados.

Listagem 3.16: Miniprojeto, Módulo 1: modelo de dados e primeiras operações.

```

1  # ---- Modelo de dados: uma lista de ocorrências
   (dicionários) ----
2  ocorrencias = [
3      {"id": 1, "sensor": "Radar-A1", "tipo": "superfície",
4       "velocidade_kmh": 22.2, "posicao": (-22.9, -43.2)},
5      {"id": 2, "sensor": "Radar-A1", "tipo": "superfície",
6       "velocidade_kmh": 44.4, "posicao": (-23.1, -43.5)},
7      {"id": 3, "sensor": "Radar-B2", "tipo": "aéreo",
8       "velocidade_kmh": 50.0, "posicao": (-22.7, -43.0)},
9  ]
10
11 LIMITE_KMH = 40.0
12
13 def registrar(lista, sensor, tipo, velocidade, posicao):
14     """Cria uma nova ocorrência e a adiciona à lista.
15     Devolve o id."""
16     novo_id = len(lista) + 1
17     lista.append({
18         "id": novo_id, "sensor": sensor, "tipo": tipo,
19         "velocidade_kmh": velocidade, "posicao": posicao,
20     })
21     return novo_id
22
23 def listar_alertas(lista, limite):
24     """Devolve as ocorrências com velocidade acima do
25     limite."""
26     return [o for o in lista if o["velocidade_kmh"] > limite]
27
28 def sensores_distintos(lista):
29     """Devolve o conjunto de sensores presentes nas
30     ocorrências."""
31     return {o["sensor"] for o in lista}
32
33 # ---- Uso ----
34 registrar(ocorrencias, "Sonar-1", "submarino", 18.5, (-23.0,
35     -43.3))
36
37 print(f"Total de ocorrências: {len(ocorrencias)}")
38 print(f"Sensores envolvidos:

```

```

35         {sensores_distintos(ocorrencias)})")
36     print("Ocorrências em alerta:")
37     for o in listar_alertas(ocorrencias, LIMITE_KMH):
38         print(f"    #{o['id']} {o['sensor']} ->
           {o['velocidade_kmh']} km/h")

1 Total de ocorrências: 4
2 Sensores envolvidos: {'Radar-A1', 'Radar-B2', 'Sonar-1'}
3 Ocorrências em alerta:
4     #2 Radar-A1 -> 44.4 km/h
5     #3 Radar-B2 -> 50.0 km/h

```

Nota

Este código é o **ponto de partida** do miniprojeto. Ele tem, porém, uma fragilidade evidente: tudo vive apenas na memória. Ao encerrar o programa, todas as ocorrências se perdem. No próximo capítulo, daremos a esse sistema a capacidade de *persistir* seus dados – gravá-los em arquivo e em banco de dados –, para que sobrevivam entre execuções. É o tema do Módulo 2.

3.8 O caminho à frente

Com este capítulo, encerramos os fundamentos da linguagem em si: a sintaxe (Capítulo 2) e as estruturas para organizar dados (Capítulo 3). Daqui em diante, passamos a construir software mais robusto e completo. O Capítulo 4 ataca a primeira limitação do nosso miniprojeto – a volatilidade dos dados – ensinando a ler e gravar arquivos, a usar um banco de dados SQLite, a extrair informação de texto com expressões regulares e a tratar erros de forma controlada. O sistema deixará de ser um exercício de

memória e passará a guardar, de fato, o que registra.

Resumo do capítulo

- A **lista** (`[]`) é a estrutura de trabalho: sequência ordenada e mutável, acessada por **índice** (a partir de 0), fatiável, com métodos como `append`, `insert`, `remove`, `pop` e `sort`.
- A **compreensão de lista** (`[f(x) for x in lista if cond]`) constrói e filtra coleções em uma linha legível.
- A **tupla** (`()`) é uma sequência **imutável** – ideal para dados que não devem mudar, como coordenadas – e suporta **desempacotamento**.
- O **dicionário** (`{chave: valor}`) guarda dados acessados por **rótulo**; use `.get` para acesso seguro e `.items()` para percorrer.
- O **conjunto** (`set`) guarda itens **únicos** e responde a **pertencimento** e operações de conjunto com eficiência.
- A combinação **lista de dicionários** representa uma “tabela” de dados e é a base do miniprojeto. No **Módulo 1**, modelamos as ocorrências em memória e escrevemos as primeiras operações.

Armadilhas comuns

- **Esquecer que o índice começa em zero.** Em uma lista de n elementos, os índices vão de 0 a $n - 1$. Acessar além disso gera `IndexError`.
- **Atribuir o resultado de `sort()`.** Métodos que alteram a lista no lugar devolvem `None`. Use `lista.sort()` ou, para uma cópia, `sorted(lista)`.
- **Acessar chave inexistente com colchetes.** Provoca `KeyError`; prefira `.get(chave, padrão)` quando a presença não é garantida.
- **Tentar alterar uma tupla.** Tuplas são imutáveis; se o dado precisa

mudar, use uma lista.

- **Confundir o limite do fatiamento.** Em `lista[2:5]`, o índice 5 é *exclusivo* – entram apenas 2, 3 e 4.
- **Esperar ordem em um conjunto.** Conjuntos não têm ordem definida; se a ordem importa, use uma lista.

Exercícios

Essencial — fixação, com solução guiada no repositório

Exercício 3.1 Crie uma lista com o nome de cinco meios de uma frota. Imprima o primeiro e o último usando índices (incluindo um índice negativo). Em seguida, adicione um sexto meio com `append` e imprima o total com `len`.

Exercício 3.2 Dada a lista `leituras = [31, 47, 22, 55, 40, 38]` (em km/h), use **fatiamento** para imprimir as três primeiras leituras e, separadamente, as duas últimas.

Exercício 3.3 Crie um **dicionário** que represente um sensor, com as chaves `nome`, `tipo` e `ativo`. Imprima o tipo do sensor e, em seguida, altere o valor de `ativo` para `False`.

Tático — aplicação a um problema semelhante

Exercício 3.4 A partir de `leituras = [31, 47, 22, 55, 40, 38]`, use uma **compreensão de lista** para criar uma nova lista contendo apenas as leituras acima de 40 km/h. Depois, crie outra que contenha todas as leituras convertidas de km/h para nós (divida por 1,852).

Exercício 3.5 Dada uma **lista de dicionários** representando meios, cada um com as chaves `nome` e `tipo`, escreva um trecho que use um **conjunto** para descobrir e imprimir quantos tipos *distintos* de meio existem na frota.

Exercício 3.6 Escreva uma função `resumo(leituras)` que receba uma lista de velocidades e devolva uma **tupla** com (menor, maior, média). Use desempacotamento ao chamá-la para imprimir os três valores. *Dica:* as funções embutidas `min`, `max`, `sum` e `len` ajudam.

Estratégico — extensão criativa

Exercício 3.7 Estenda o miniprojeto da Listagem 3.16 com uma nova função `contar_por_tipo(lista)` que devolva um **dicionário** em que cada chave é um tipo de ocorrência (“superfície”, “aéreo” etc.) e cada valor é quantas ocorrências há daquele tipo. *Dica:* percorra a lista e use `.get(tipo, 0) + 1` para acumular as contagens.

Exercício 3.8 Ainda no miniprojeto, escreva uma função `ocorrencias_do_sensor(sensor)` que devolva apenas as ocorrências de um sensor específico, e uma função `velocidade_media(lista)` que calcule a velocidade média de uma lista de ocorrências. Combine-as para responder: qual a velocidade média das ocorrências do sensor “Radar-A1”? Reflita sobre como essas pequenas funções, somadas, começam a formar um verdadeiro sistema de consulta.

Fim da amostra

Aqui termina a amostra do Módulo 1. Os módulos seguintes tratam de persistência de dados, expressões regulares e tratamento de erros; programação orientada a objetos; bibliotecas científicas (*NumPy*, *pandas*, *Matplotlib*); e um projeto final integrador de apoio à decisão.

Os *notebooks* de aula continuam disponíveis no repositório do projeto.